

脑机接口蓝牙通信协议 — 上位机开发指南

适用设备：植入式刺激器 (Device Type 0x00)
上位机平台：Android / iOS / Windows / macOS (需支持 BLE 4.2+)

一、通信架构概述

1.1 BLE 连接方式

植入式刺激器作为 BLE Peripheral (从设备)，上位机作为 BLE Central (主设备)。

- 蓝牙版本要求：** BLE 4.2 及以上，支持 LE Secure Connections (LESC)
- 配对方式：** LESEC Just Works (无 PIN 码配对)
- 首选 PHY：** LE 2M (2Mbps 物理层)
- MTU：** 最大 247 bytes
- GAP Data Length：** 251 bytes

1.2 连接参数

上位机应尽量满足设备端提供的 Preferred Connection Parameters：

参数	推荐值
最小连接间隔	15 ms
最大连接间隔	15 ms
Slave Latency	30
连接超时	4000 ms

说明： 15ms 连接间隔 + LE 2M PHY 可最大化吞吐量，降低实时数据延迟。

1.3 GATT Service / Characteristic 结构

设备暴露两个 GATT Service：

DBS Service (主业务服务，UUID 见下表)：

Characteristic	UUID	Properties	用途
Write Config	XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX (脱敏)	Write	上位机下发控制/查询指令
Device Notify	XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX (脱敏)	Notify	设备返回响应 + 主动上报状态
Stream Data	XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX (脱敏)	Notify	实时感测数据上传 (0xC0)

Characteristic	UUID	Properties	用途
Storage Data	XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX (脱敏)	Notify	存储数据上传 (0xE0)

Secure DFU Service (固件升级, UUID 0xFE59) :

Characteristic	UUID	Properties	用途
Buttonless DFU	XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX (脱敏)	Indicate, Write	Nordic DFU 固件升级

1.4 上下行数据通道总结

上位机 → 设备:	write Config (GATT write / write Command)
设备 → 上位机:	Device Notify (GATT Notify) – 配置响应、状态上报
	Stream Data (GATT Notify) – 实时感测数据 (CMD 0xC0)
	Storage Data (GATT Notify) – 存储数据上传 (CMD 0xE0)

- Write Config 支持 Write Request (设备返回 GATT 错误码) 和 Write Command (无响应) 。
- 正常通信建议使用 Write Request, 上位机可感知 GATT 层错误 (0x80~0x9F) 。

二、数据包格式详解

2.1 整体帧结构 (PDU Frame)

上位机发送和接收的每一帧数据均遵循以下结构 (所有多字节字段为大端序) :

-----+-----+-----+-----+-----
----+
Frame Control Data Header Data (Optional)
(2 bytes) (8 bytes) (0~232 bytes) Auth Signature
(28 bytes)
-----+-----+-----+-----+-----

加密模式下, Data Header + Data 区域被加密, 并额外附加 28 字节 Authentication Signature。非加密模式下无此字段。

2.2 Frame Control (帧控制, 2 字节)

Byte 0: Preamble (0xA5, 固定)
Byte 1: [Rolling Counter(4bit)] [Encryption(1bit)] [AckReq(1bit)]
[Reserved(2bit)]

字段	位宽	说明
Preamble	8 bits	固定 0xA5, 帧头标识。不匹配则丢弃整帧

字段	位宽	说明
Rolling Counter	4 bits	滚动计数器，每帧自增 1，达到 15 后翻转为 0。上位机和设备各自独立维护。收到连续相同计数器值的包，丢弃后面的（防重放）
Encryption Enabled	1 bit	0 = 加密关闭；1 = 加密开启（Data Header + Data 区域加密，末尾附 28 字节签名）
Acknowledge Request	1 bit	0 = 无需应答；1 = 期望对方回复
Reserved	2 bits	保留，填 0

2.3 Data Header（数据头部，8 字节）

字段	字节数	说明
Timestamp (ms)	6	Unix 毫秒时间戳。前 4 字节存秒数，后 2 字节存毫秒数。 大端序传输 ，加密后用于对抗重放攻击
Total Packet	1	总分包数（1~255）
Current Packet	1	当前分包序号（1~255）
Data Length	1	Data 区长度（不含 Frame Control / Data Header / Auth Signature）。未加密时 0~232，加密时 0~204（因为 28 字节签名占用空间）

2.4 Data（数据区，变长）

Data 区由一个或多个 PDU（Protocol Data Unit）组成。

2.5 PDU 格式

+-----+-----+-----+
Opcode PDU Len PDU Data
1 byte 2 bytes n bytes
+-----+-----+-----+

字段	字节数	说明
Opcode	1	原子操作码，每条命令的最小执行单元
PDU Length	2	PDU Data 长度（不含 opcode 和自身）。大端序
PDU Data	n	协议数据内容。若一个 PDU 跨蓝牙包，在下个分包中继续传，不重复 opcode 和 PDU Length

关键特性： 多个 PDU 可以组合在同一个 Data 区中发送（除非文档特别说明禁止组合，如 CMD 0x02 安全状态配置）。

2.6 Authentication Signature（可选，28 字节）

仅加密模式下存在：

字段	字节数	说明
MAC	16	ChaCha20-Poly1305 消息认证码
Nonce	12	加密 nonce

2.7 Frame Control 字节编码速查

上位机构建 Frame Control 第二字节的方法：

```
Byte1 = (RollingCounter << 4) | (Encryption << 3) | (AckReq << 2) | Reserved

示例（非加密，需应答，RC=0）：
Byte1 = (0 << 4) | (0 << 3) | (1 << 2) | 0 = 0x04
全帧头 = [0xA5, 0x04]
```

2.8 大端序编码规则

所有多字节字段使用大端序（Big-Endian / Network Byte Order）传输。 包括但不限于：

- Timestamp（6 字节）
- PDU Length（2 字节）
- PDU Data 中的 u16 / u32 / float 等

上位机构建数据包时，务必使用大端序编码（如 Java 的 `ByteBuffer.order(ByteOrder.BIG_ENDIAN)`、Python 的 `struct.pack('>H', value)`）。

2.9 分包传输

当 Data 区超过单包承载能力时，需要分包：

- 每个分包都带完整 Frame Control + Data Header
- Total Packet 填总分包数，Current Packet 填当前序号
- PDU 可以跨越分包边界：上一个包的 PDU Data 未完成部分，在下一个包的 Data 区继续，不重复 opcode 和 PDU Length

三、安全认证流程

3.1 整体流程图



```

|
| --- CMD 0x01 Opcode 0x00 (安全状态查询) --->|
|<-- has_private_key + is_app_trusted -----|
|
| 分支 A: 设备无私钥
| --- CMD 0x02 Opcode 0x00 (传输私钥) ----->|
| [Secret Key 32B + Adata 4B]
|<-- 0x00 Success/Fail -----|
|
| 分支 B: App 未被信任
| --- CMD 0x02 Opcode 0x01 (App ECDH公钥) --->|
| [App Public Key 64B]
|<-- Device ECDH Public Key 64B -----|
|
| App: 计算 Shared Secret
|       = ECDH(App Private, Device Public)
|
| Device: 计算 Shared Secret
|       = ECDH(Device Private, App Public)
|
| App: Digest = HMAC-SHA256(Shared, Secret)
|
| --- CMD 0x02 Opcode 0x02 (发送摘要) ----->|
| [HMAC-SHA256 Digest 32B]
|
| Device: 用 Shared Secret 解密 + 对比 Secret
|<-- 0x00 Success / 0x01 Fail -----|
|
| Device: App 加入白名单
|
|===== 安全通道已建立 =====|
|
| 后续通信: ChaCha20-Poly1305 AEAD 加解密
| (Encryption Enabled = 1)

```

3.2 安全状态查询 (CMD 0x01 Opcode 0x00)

方向: App → Dev (查询)

PDU: Opcode=0x00, Length=0, Data=空

响应 (Dev → App):

字节	说明
Byte 0	bit 0: 0=设备无密钥, 1=设备有密钥
	bit 1: 0=App 未受信任, 1=App 已受信任
	bit 2-7: 保留

上位机判断逻辑:

```
if has_private_key == 0:
    执行 CMD 0x02 Opcode 0x00 (传输私钥)
elif is_app_trusted == 0:
    执行 CMD 0x02 Opcode 0x01 (ECDH 公钥交换) → Opcode 0x02 (验证摘要)
else:
    安全通道已就绪, 直接进入业务通信
```

3.3 传输私钥 (CMD 0x02 Opcode 0x00)

前提: 设备无密钥 → 需要设置 (整个设备生命周期仅执行一次)

方向: App → Dev

PDU: Opcode=0x00, Length=36

字段	字节数	说明
Secret Key	32	上位机生成的 32 字节随机密钥, 用作后续 HMAC 的 Secret。上位机必须妥善持久化存储
Adata	4	附加认证数据

响应: Opcode=0x00, Length=1, 0=成功, 1=失败

3.4 ECDH 公钥交换 (CMD 0x02 Opcode 0x01)

前提: 设备已有私钥但 App 未受信任

方向: App → Dev

PDU: Opcode=0x01, Length=64, Data=App 的 ECDH 公钥 (64 字节, secp256r1 曲线)

响应: Opcode=0x01, Length=64, Data=设备的 ECDH 公钥

上位机处理:

```
app_private_key = 随机生成 secp256r1 私钥
app_public_key = ECDH 公钥派生
发送 app_public_key → 设备
收到 device_public_key →
shared_secret = ECDH(app_private_key, device_public_key)
```

3.5 摘要验证 (CMD 0x02 Opcode 0x02)

方向: App → Dev

PDU: Opcode=0x02, Length=32

Data:

```
digest = HMAC-SHA256(key=shared_secret, message=secret_key)
```

即: 用 ECDH 共享密钥作为 HMAC 密钥, 对 Secret Key 计算 SHA256 摘要。

响应: Opcode=0x02, Length=1, 0=验证通过 (App 加入白名单), 1=失败

3.6 数据加密 (ChaCha20-Poly1305 AEAD)

认证完成后，后续业务通信默认启用加密。

- **加密算法**：ChaCha20-Poly1305 AEAD
- **加密范围**：Data Header + Data 区域
- **额外产物**：28 字节 Authentication Signature (MAC 16B + Nonce 12B)
- **计数器**：设备端 `dbb_get_counter()` 在每次 BLE 连接初始化时重置

不需加密的例外：

- CMD 0x01 (安全状态查询)
- CMD 0x02 (安全状态配置)
- CMD 0xC0 (实时数据上传, Stream Data)

上位机实现要点：

- 加密模式下，Frame Control 的 Encryption Enabled 位 = 1
- 解密时先验证 MAC (防篡改)，再解密 Data Header + Data
- Nonce 由发送方生成，附在 Auth Signature 中

四、请求-应答模式

4.1 构建请求包流程

1. 确定 Command (1 字节)
2. 构建 PDU: Opcode (1B) + PDU Length (2B, 大端) + PDU Data (Length 字节)
3. 拼接多个 PDU 到 Data 区
4. 判断是否需要分包：
 - 加密模式: Data 区 ≤ 204 字节 / 包
 - 非加密模式: Data 区 ≤ 232 字节 / 包
5. 构建每包：
 - Frame Control: $0xA5 + (RC \ll 4 \mid Enc \ll 3 \mid AckReq \ll 2 \mid 0)$
 - Data Header: Timestamp + Total + Current + DataLen
 - Data 区
 - 若加密: Auth Signature (28B)
6. 通过 GATT Write 发送到 Write Config Characteristic

4.2 解析响应包流程

1. 从 Device Notify Characteristic 的 GATT Notify 中接收数据
2. 检查 Preamble (第 1 字节必须是 0xA5)，不匹配则丢弃
3. 检查 Rolling Counter，与上次相同则丢弃 (防重放)
4. 提取 Encryption Enabled 位：
 - 若 = 1: 验证 MAC → 解密 Data Header + Data → 提取 PDU
 - 若 = 0: 直接提取 PDU
5. 检查分包：
 - 若 Total > 1: 缓存各分包，全部收齐后拼接 Data 区再解析 PDU
6. 从 Data 区按序解析 PDU: 读 Opcode → 读 PDU Length → 读 PDU Data
7. 根据 Command + Opcode 分发到业务处理

4.3 GATT 层错误码

设备通过 Write Request 的 GATT Status Code 返回处理结果：

GATT Status	含义
0x00	操作成功
0x80	Format Error：数据格式/长度/量程错误
0x81	Flash Error：存储错误，掉电恢复
0x82	State Error：状态不允许当前操作

4.4 超时处理

上位机应实现超时重试机制：

- 请求发出后启动定时器（建议 3~5 秒）
- 超时未收到响应则重试（最多 2 次）
- 若仍无响应，断开连接并提示用户"设备无响应"
- 注意 Rolling Counter 同步：重试前确认计数器值未被消耗

五、各协议详细使用说明

实现状态标记：✅ 完整实现 | ⚠️ 部分实现 | ❌ 空桩/Stub（代码未实现，上位机调用会直接返回 DBS_ERR_NONE，无实际效果）

5.1 CMD 0x00 — 开发者配置 ⚠️ 部分

用途：触发恢复出厂、重启、清绑定等系统级操作。

App → Dev：PDU 长度均为 0（无 Data）

Opcode	操作	说明
0x00	Factory Reset	注意： 代码中 RECOVER 阶段仅处理了删密钥，其余清理可能未完整执行。APP 端须同步解除绑定
0x01	Reboot	设备重启
0x02	Remove Bonding	清除绑定并重启。APP 端须同步解除绑定
0x03	Remove Private Key	清除密钥并重启
0x04	Clear Flash	清除 Flash（配置+运行状态）并重启。 注意： Flash 持久化本身未实现，此操作可能无实际效果

响应：设备处理后断开连接（触发恢复流程），无直接 PDU 响应。

代码现状：标志位设置完整，但 RECOVER 事件处理仅覆盖了 REMOVE_KEY（调用 `fds_file_delete`），其余 4 项的实际执行未在本模块验证到。

5.2 CMD 0x01 — 安全状态查询 完整

用途：查询设备安全状态（是否已设置密钥、App 是否已受信任）。

App → Dev：

Opcode	Length	Data
0x00	0	空

Dev → App：

Opcode	Length	Data
0x00	1	Byte 0: bit0=has_private_key, bit1=is_app_trusted

示例响应解析：

```
收到 Data = 0x03
  bit0 = 1 → 设备有密钥
  bit1 = 1 → App 已受信任
  → 跳过安全配置，直接进入业务通信
```

5.3 CMD 0x02 — 安全状态配置 完整

用途：完成安全认证的完整流程（传密钥 → ECDH 交换 → 摘要验证）。

重要：本 Command 禁止多个 opcode 组合在同一个数据包中，必须逐条发送。

5.3.1 Opcode 0x00 — 传输私钥

App → Dev（整个设备生命周期仅执行一次）：

Opcode	Length	Data
0x00	36	Secret Key (32B) + Adata (4B)

构建示例（伪代码）：

```
secret_key = generate_random_bytes(32) // 上位机持久化存储
adata = b'\x00\x00\x00\x00'          // 或自定义
pdu_data = secret_key + adata
pdu = pack_pdu(0x00, pdu_data)        // Length=36
```

Dev → App：Opcode=0x00, Length=1, Data=0（成功） / 1（失败）

5.3.2 Opcode 0x01 — ECDH 公钥交换

App → Dev:

Opcode	Length	Data
0x01	64	App ECDH Public Key (secp256r1, 64 字节)

Dev → App: Opcode=0x01, Length=64, Data=设备的 ECDH 公钥

5.3.3 Opcode 0x02 — 摘要验证

App → Dev:

Opcode	Length	Data
0x02	32	HMAC-SHA256(shared_secret, secret_key)

Dev → App: Opcode=0x02, Length=1, Data=0 (通过) / 1 (失败)

5.4 CMD 0x03 — 本机状态查询 完整

用途: 查询设备硬件/固件/电池/温度等本机状态。

App → Dev: 无需指定 opcode, 设备会上报所有状态 (主动上报时仅上报变化项)。

Dev → App (查询响应, 返回全部实现项):

Opcode	Length	Data	类型	主动上报
0x00	4	Hardware Version	string "Va.b"	否
0x01	6	Firmware Version	string "yyyyMMdd"	否
0x03	1	Battery %	u8 (0~100)	变化时
0x04	1	Charge Indicator	u8 (0/1)	变化时
0x05	2	Charge Current	u16 (mA)	变化时
0x06	2	Charge Voltage	u16 (mV)	变化时
0x07	6	Activate Timestamp	u32秒 + u16毫秒	否
0x08	2	Device Temperature	i16 (0.1°C)	变化时
0x09	2	Battery Cycle	u16 (次)	否
0x0E	1	Device Humidity	u8 (0~100 %RH)	变化时
0x0F	2	Battery Voltage	u16	变化时

代码新增 opcode (协议文档中未列出, 但代码已完整实现):

- 0x0E (设备湿度) — u8, 0~100 %RH, 需传感器进程填充
- 0x0F (电池电压) — u16

示例响应解析：

```
收到多个 PDU，逐个解析...
Opcode=0x03, Len=1, Data=0x55 → 电量 85%
Opcode=0x08, Len=2, Data=0x00FA → 温度 25.0°C (250 * 0.1)
Opcode=0x0E, Len=1, Data=0x2D → 湿度 45 %RH
```

5.5 CMD 0x04 — 用户数据查询 ✔ 完整

用途：查询患者信息、电极配置等用户数据。

App → Dev：同 CMD 0x03，无需指定 opcode。

Dev → App：

Opcode	Length	Data	类型
0x00	4	Patient ID	u32 (0=未设置)
0x01	<32	Patient Name	UTF-8 string
0x02	1	Gender	u8 (0=未设置, 1=男, 2=女)
0x03	4	Birthday	u32 (Unix timestamp 秒)
0x04	4	Operation Access	u32 (bit 权限)
0x05	1	Stimulate Location	u8 (0=左胸, 1=右胸, 2=其他)
0x06	4	Electrode Type	u8×4 (每位置 0=关, 其他=开)
0x07	2	Channel Index	u16 (bit0~15 = 通道1~16)
0x08	<128	Line Name	UTF-8 string
0x09	1	Charge Speed	u8 (0=高, 1=中, 2=低)

5.6 CMD 0x05 — 用户数据配置 ✔ 完整 (Flash 持久化未实现)

用途：配置患者信息等用户数据。

App → Dev：Opcode 参考 CMD 0x04 (0x00~0x09)，Length 和 Data 同对应查询项的格式。

示例：

```
配置患者 ID 为 12345：
Opcode=0x00, Length=4, Data=0x00003039 (u32 大端)

配置性别为男：
Opcode=0x02, Length=1, Data=0x01
```

响应：Dev → App 返回确认响应。

⚠ 注意：代码中 Flash 持久化未实现（flash_process 为空壳），配置修改仅在 RAM 中，设备断电后丢失。上位机需要在每次连接后重新下发配置。

5.7 CMD 0x06 — 感测参数查询 ✔ 完整

用途：查询实时感测和 LFP 感测的通道、频率、滤波器等参数。

Dev → App：

Opcode	Length	Data	类型	默认值
0x00	2	Live Sampling Channel Index	u16	0
0x01	2	Live Sampling Frequency	u16	1000 (Hz)
0x02	8	Live Filter Bandwidth	float×2 (低+高)	0.1, 250.0
0x03	24	Spectrogram Band Marker	binary	全 0
0x04	2	LFP Sampling Channel Index	u16	0
0x05	2	LFP Sampling Frequency	u16	1000 (Hz)
0x06	8	LFP Filter Bandwidth	float×2	0.1, 250.0

5.8 CMD 0x07 — 感测参数配置 ✔ 完整 (Flash 持久化未实现)

用途：配置感测参数。Opcode 0x00~0x02，格式参考 CMD 0x06。

同 CMD 0x05，Flash 持久化未实现，掉电丢失。

5.9 CMD 0x08 — 刺激参数查询 ✔ 完整

用途：查询指定编程组的刺激参数。

App → Dev：发送 Opcode=0x00 指定查询的编程组号（1~4），Length=1。

Dev → App（遍历指定组全部参数）：

Opcode	Length	Data	类型
0x00	1	Stimulate Group	u8 (1~4)
0x01	1	Method	u8 (0=恒流, 1=恒压)
0x02	2	Frequency	u16 (2~250 Hz)
0x03	1	Pair Count (n)	u8 (1~8)
0x04	2×n	Pole Numbers	u8+u8 每对
0x05	4×n	Current/Voltage	float (mA/V)
0x06	4×n	Pulse Width	u16+u16 (μs, 20~450)
0x07	2	Interval (Symmetric Poles)	u16 (μs, 默认 60)
0x08	2	Soft Start	u16 (秒, 0/1/2/4/8)
0x09	4	Periodic Start	u16刺激+u16间歇 (秒)
0x0A	1	Active Discharge	u8 (0=关, 1=开)

Opcode	Length	Data	类型
0x0B	<32	Group Name	UTF-8 string
0x0C	1	Program Set	u8 (0=程序组1, 1=程序组2)

示例： 查询编程组 1 的参数

```
发送：CMD 0x08
PDU：Opcode=0x00, Len=1, Data=[0x01]
→ 设备返回该组的全部 13 个 opcode 响应
```

5.10 CMD 0x09 / 0x11 — 刺激参数配置 / 感测刺激配置 ✓ 完整 (Flash 未实现)

CMD 0x09： 配置编程组 1~4 的刺激参数。
CMD 0x11： 配置编程组 0（感测刺激参数）。
两者共用同一个处理函数 `dbs_cmd_stim_para_config()`。

App → Dev： Opcode 参考 CMD 0x08，先发 0x00 选组号，再发其他配置项。

新增 opcode（代码实现，协议文档未列出）：

- 0x0D（微调模式）：Length=1，Data=1/0 开/关。开启后不触发 `PROCESS_EVENT_RUN_STATUS`，用于安全的参数微调。

Flash 持久化未实现。

5.11 CMD 0x0A / 0x0B — 控制参数查询/配置 ✗ 空桩

状态： 两个函数体均为 `return DBS_ERR_NONE;`，零业务逻辑。

上位机调用将收到成功响应但无任何实际效果。**避免使用这两个 Command。**

5.12 CMD 0x0C — 运行模式查询 ✓ 完整

用途： 查询设备当前的运行状态（各开关状态、当前编程组、定时计划等）。

App → Dev： 无需指定 opcode，设备上报全部。

Dev → App：

Opcode	Length	Data	类型
0x00	2	Switch Bitmask	u16（见下表）
0x01	1	Active Stimulate Group	u8 (0~4)
0x05	1+5×n	Schedule	u8组数 + n×(Start u16 + Stop u16 + Group u8)
0x06	1	Enable Scheduled Switch	u8
0x07	1	Live Sample Start	u8

Opcode	Length	Data	类型
0x08	1	Stimulate Start	u8
0x09	1	Impedance Start	u8
0x0B	1	LFP Sample Start	u8
0x0F	1	Motion Sample Start	u8（代码新增）

综合开关 Bitmask (Opcode 0x00):

Bit	含义
bit 0	Live Sample Start（实时感测）
bit 1	Stimulate Start（刺激器）
bit 2	Impedance Start（阻抗）
bit 3	Control Start（控制器）
bit 4	LFP Sample Start（LFP 感测）
bit 5~15	保留

5.13 CMD 0x0D — 运行模式配置 完整

用途：配置设备运行状态（开关刺激/感测/阻抗/定时切换等）。

App → Dev（关键 opcode）：

Opcode	操作	说明
0x00	综合开关	Length=2, Data=u16 Bitmask（格式同 CMD 0x0C Opcode 0x00）
0x01	当前刺激编程组	Length=1, Data=组号(0~4)。自动关刺激→切换→可选重开
0x04	Shipping 模式	Length=1, 1=进入运输模式
0x06	定时切换开关	Length=1, 1=开/0=关。刺激运行时不可更改
0x07	实时感测开关	Length=1
0x08	刺激器开关	Length=1
0x09	阻抗测量开关	Length=1
0x0B	LFP 感测开关	Length=1
0x0D	定时切换扩展	含 enable_group[] 独立开关（代码新增）
0x0E	一键撤回	恢复患者权限+全部刺激参数+定时计划（代码新增）
0x0F	Motion 感测开关	与实时感测互斥（代码新增）

安全互斥约束（代码已实现）：

- 测阻抗或充电时禁止开启感测（返回 DBS_ERR_STATE）
- Motion 感测与实时感测互斥
- 刺激运行时不可更改定时切换

5.14 CMD 0x0E — 阻抗测试选项 ⚠ 部分

App → Dev（配置方向，代码已实现）：

Opcode	Length	Data	类型	默认值
0x01	2	Impedance Channel Index	u16, bit 通道选择	0xFFFF
0x02	4	Impedance Check Interval	u32 (1800~86400 秒)	86400
0x04	1	Sync Impedance Results	u8 (0x01=开始同步)	0x01

Dev → App（查询/上报方向）：

- 0x00（单端阻抗结果）、0x11（差分阻抗结果）不在本模块处理
- 阻抗测试结果由 `ps_adc.c` 通过 CMD 0xC0（Stream Data）发送
- 同步依赖 `flash_process`，但 `flash_process` 为空壳，历史阻抗同步实际不执行

阻抗结果数据格式 (0x00 / 0x11)：

字段	字节数	说明
Timestamp	6	测试时间
Channel Bitmask	2	通道开启表
Datapoint	1	测试点数
Value Array	Active_Channels × Datapoint × 2	u16 数组

0x00（单端）：值顺序为 ch1dp1, ch1dp2, ..., ch2dp1, ..., ch16dpN

0x11（差分）：顺序为 ch1-ch2, ch1-ch3, ..., ch15-ch16

5.15 CMD 0x0F — 保留 ❌ 空桩

函数体 `return DBS_ERR_NONE;`。避免使用。

5.16 CMD 0x10 — 存储数据选项 ⚠ 部分

App → Dev：

Opcode	Length	Data	说明
0x00	1	0x00 = 开始同步	设备上传全部缓存数据

Dev → App（响应上报）：

Opcode	Length	Data	说明
0x00	1	0x01 = 同步结束	全部上传完毕
0x01	1	0x00 开始 / 0x01 结束	神经标记物同步
0x02	1	0x00 开始 / 0x01 结束	电源事件同步

代码现状：标志位和状态检查逻辑完整，但 `flash_process` 为空壳，**实际上传不会发生**。调用 0x00 后会设置 `flash_syncing` 标志并 post 事件，但事件无消费者。

上位机如需测试，建议先发送 Opcode 0x00，等待 Opcode 0x00 响应（同步结束），期间应为 CMD 0xE0 数据做好准备。但由于代码未实现，目前不会收到实际数据。

六、数据上传处理

6.1 CMD 0xC0 — 实时感测数据上传 完整

通道：Stream Data Characteristic (Notify)

用途：设备定时（由 `adc_process` 触发）将实时感测数据推送到上位机。

加密：默认关闭（Stream Data 不加解密，以降低延迟）

PDU 格式：

字段	字节数	说明
Opcode	1	0x00
Length	2	PDU 数据长度（大端）
Timestamp	6	第一个采样点的时间戳（前 4 秒 + 后 2 毫秒）
Channel Bitmask	2	bit0~15 = 通道1~16, 1=开启
Datapoint	1	采样点数 N
Value Array	ActiveCh × N × 2	i16 数组（大端），顺序：ch1dp1, ch2dp1, ..., ch16dp1, ch1dp2, ..., ch16dpN

上位机解析伪代码：

```
active_channels = count_bits(channel_bitmask) // 统计开启通道数
total_values = active_channels * datapoint
expected_data_len = 6 + 2 + 1 + total_values * 2 // 验证 Length

values = []
for dp in range(datapoint):
    for ch in range(16):
        if channel_bitmask & (1 << ch):
            value = read_i16_be(data, offset)
            values[(ch+1, dp)] = value
            offset += 2
```


行为注意：

- 设备在发送 Stream Data 时会**暂停广播**以降低丢包率
- Notify 不带 ack，丢包不重传
- 上位机应在连接建立后通过 CMD 0x0D Opcode 0x07 开启实时感测

6.2 CMD 0xE0 — 存储数据上传 完整（数据打包逻辑完整，上游依赖 Flash 同步）

通道：Storage Data Characteristic（Notify）

用途：设备将缓存的神经标记物和电源事件数据推送到上位机。

6.2.1 神经标记物 (Opcode 0x00)

字段	字节数	说明
Opcode	1	0x00
Length	2	PDU 数据长度
Datapoint	1	数据点数 N
Value Array	N × (4+4)	N 组 (Timestamp u32 秒 + Biomarker float)，全部大端

解析：

```
for i in range(datapoint):
    timestamp = read_u32_be(data, offset)
    biomarker = read_float_be(data, offset+4)
    offset += 8
```

6.2.2 电源事件 (Opcode 0x01)

字段	字节数	说明
Opcode	1	0x01
Length	2	PDU 数据长度
Datapoint	1	事件点数 N
Value Array	N × (4+1)	N 组 (Timestamp u32 秒 + PowerEvent u8)，全部大端

电源事件类型：

PowerEvent	含义
0x00	进入休眠模式

容量限制：

- 神经标记物：最多 84 条，同步后不删除
- 电源事件：最多 10 条，同步后不删除

⚠ 上位机需注意：CMD 0xE0 数据的实际产生依赖 Flash 同步流程（CMD 0x10），由于 `flash_process` 为空壳，当前代码中存储数据实际上传不可用。上位机仍应实现解析逻辑以备后续固件完善。

七、典型 workflow

7.1 完整连接→认证→配置→刺激控制→数据接收流程

阶段 1：BLE 连接

- └ 扫描设备（按 Service UUID 过滤或 Manufacturer Data 识别）
- └ 读取广播包获取设备状态（电量、温度、充电状态等）
- └ 发起 BLE 连接
- └ 设置 MTU = 247
- └ 设置 PHY = LE 2M
- └ LESC Just Works 配对
- └ 启用 Device Notify / Stream Data / Storage Data 的 CCCD

阶段 2：安全认证

- └ 发送 CMD 0x01 opcode 0x00（安全状态查询）
- └ 根据 has_private_key / is_app_trusted 判断：
 - └ 无密钥 → CMD 0x02 opcode 0x00（传密钥）
 - └ 未信任 → CMD 0x02 opcode 0x01（ECDH）→ opcode 0x02（摘要验证）
 - └ 已信任 → 跳过
- └ 安全通道建立，后续通信启用加密

阶段 3：状态查询与配置同步

- └ CMD 0x03（本机状态）→ 记录硬件/固件版本、电量、温度、湿度等
- └ CMD 0x0C（运行模式）→ 记录当前开关状态
- └ CMD 0x04（用户数据查询）→ 获取患者信息
 - └ 如需要，CMD 0x05 重新配置
- └ CMD 0x06（感测参数查询）
 - └ 如需要，CMD 0x07 重新配置
- └ CMD 0x08（刺激参数查询，逐组 1~4）
 - └ 如需要，CMD 0x09 重新配置
- └ CMD 0x0E（阻抗选项查询）

阶段 4：刺激控制

- └ CMD 0x09/0x11（配置刺激参数）
 - └ 先 0x0D 进入微调模式（可选）
- └ CMD 0x0D opcode 0x01（选择当前编程组）
- └ CMD 0x0D opcode 0x08（开启刺激器）
 - └ 或 CMD 0x0D opcode 0x00 综合设置
- └ 监控刺激状态变化（CMD 0x0C 主动上报）

阶段 5：数据接收

- └ 实时感测：
 - └ CMD 0x0D opcode 0x07（开启实时感测）
 - └ 监听 Stream Data Characteristic
 - └ 解析 CMD 0xC0 数据包
- └ 存储数据同步（当前代码不可用）：
 - └ CMD 0x10 opcode 0x00（开始同步）
 - └ 监听 Storage Data Characteristic
 - └ 解析 CMD 0xE0 数据包

└─ CMD 0x0D Opcode 0x07=0（关闭实时感测）

阶段 6：断开

└─ 停止刺激 → CMD 0x0D Opcode 0x08=0

└─ 关闭感测 → CMD 0x0D Opcode 0x07=0

└─ 断开 BLE 连接

└─ 注意：实时感测/LFP 感测在断开后自动关闭

7.2 编程组定时切换 workflow

1. CMD 0x09（配置各编程组 1~4 的刺激参数）

2. CMD 0x0D Opcode 0x05（设置定时计划）

格式：ScheduleCount (u8) + nx[Start(u16 分钟) + Stop(u16 分钟) + Group(u8)]

示例：2 组定时

Group1: 08:00-12:00 用编程组 1

Group2: 14:00-18:00 用编程组 2

→ PDU Data = [0x02, 0x01E0, 0x02D0, 0x01, 0x0348, 0x0438, 0x02]

3. CMD 0x0D Opcode 0x06=1（开启定时切换）

4. 设备自动按计划切换，上位机可通过 CMD 0x0C 查询当前编程组

八、注意事项

8.1 空桩协议 — 避免使用

以下 Command 在代码中为纯占位函数（return DBS_ERR_NONE），无任何业务逻辑：

Command	函数	影响
0x0A	控制参数查询	返回成功但无数据
0x0B	控制参数配置	返回成功但无效果
0x0F	保留	返回成功但无效果

上位机应避免使用这些 Command，协议文档中 CMD 0x0A/0x0B 也标记为 TBD。

8.2 Flash 持久化全局未实现

ps_flash.c 的 flash_process 为空壳（PROCESS_YIELD() 死循环），导致：

- CMD 0x05 / 0x07 / 0x09 / 0x11 / 0x0D / 0x0E 的配置修改仅在 RAM 中，设备掉电后丢失
- CMD 0x10 的存储数据同步实际上传不会发生
- CMD 0x00 Opcode 0x04（Clear Flash）可能无实际清理效果
- CMD 0x0E Opcode 0x04（同步历史阻抗结果）无法工作

上位机对策：每次连接后假设设备为出厂默认状态，重新下发全部配置。不要在 UI 中暴露"存储数据同步"功能（当前不可用）。

8.3 大端字节序 — 必须严格遵守

所有多字节字段使用大端序传输，包括但不限于：

- 数据包头部各字段
- PDU 中的 u16 / u32 / float / i16 / 时间戳
- 时间戳结构（秒 4 字节大端 + 毫秒 2 字节大端）

常见错误：使用主机本地字节序（通常为小端序）直接写入，导致设备解析错误。

8.4 部分实现的 Command 使用限制

Command	限制
0x00	Factory Reset / Reboot / Remove Bonding / Clear Flash 在 RECOVER 阶段仅有删密钥实际执行
0x0E	仅 App→Dev 配置方向可用，Dev→App 阻抗结果上报不在本模块
0x10	标志位设置正确，但 flash_process 空壳导致同步不执行

8.5 安全相关注意事项

- **CMD 0x02 严禁多 opcode 合并** — 必须逐条发送等待响应
- **Secret Key (32 字节)** — 上位机必须在本地安全持久化存储（如 Keychain / Keystore），丢失后无法恢复
- **ECDH 私钥** — 每次认证时随机生成，无需持久化
- **加密计数器同步** — 每次 BLE 连接后设备端计数器重置，上位机也需重置

8.6 状态互斥约束

上位机下发配置时应遵守以下约束（代码已校验，违反会返回 DBS_ERR_STATE）：

- 测阻抗或充电时不可开启感测
- Motion 感测与实时感测互斥
- 刺激运行时不可更改定时切换
- 刺激运行时不可开始存储同步

8.7 广播包解析

设备在未连接或已连接状态下均发送广播包，上位机可在扫描阶段获取以下信息（小端序）：

字段	偏移	字节数	说明
Company ID	0	2	固定 0x0059
Device Type	2	1	0x00 = 植入式刺激器
Battery %	3	1	0~100
Charge Indicator	4	1	0/1

字段	偏移	字节数	说明
Charge Voltage	5	2	mV
Charge Current	7	2	mA
Device Temperature	9	2	0.1°C, i16
Device Error	11	2	错误代码
MP2710 Reg08/09/02	13~15	各 1 字节	充电管理

注意：广播包为**小端序**，与数据包的大端序不同。

8.8 实时数据传输时的特殊行为

- 设备在发送 Stream Data（CMD 0xC0）时会**暂停广播**
- Stream Data 通过 Notify 发送，**不加密，不带 ack**
- 上位机需容忍一定的丢包（Notify 无重传机制）

8.9 代码新增 opcode 对照

代码中实现了 6 个协议文档未列出的 opcode，上位机可使用：

位置	Opcode	功能
CMD 0x03	0x0E	设备湿度查询
CMD 0x03	0x0F	电池电压查询
CMD 0x09/0x11	0x0D	微调模式开关
CMD 0x0D	0x0D	定时切换独立通道开关
CMD 0x0D	0x0E	一键撤回（恢复备份参数）
CMD 0x0D	0x0F	Motion 感测开关